SO-101 + MolmoAct2: task playbook and setup checklist

Arman's SO-101 hackathon setup, 14 October 2026. This is the second guide. The first one, **docs/SO101_MolmoAct2_Guide.pdf**, has the exact camera positions with drawings and the fine-tuning plan. This one covers: which tasks to do, how to set everything up in the right order, what the new code does for you, and what to do when something goes wrong. Facts come from the MolmoAct2 paper, the model's Hugging Face page, the official LeRobot docs, the Feetech motor documentation and our own tests; estimates are marked.

Section	What it covers
1. Status	What is ready, what was tested and how, what can only be tested with the robot.
2. Your documents and files	Every PDF, command and settings line you need, in one list.
3. Setup, step by step	Plug-and-play order from power to first task, with the exact commands.
4. The best tasks	What the model was trained and tested on, a ranked list for a weak gripper, demo combos.
5. What the code does for you	Automatic cloud update, carry-to-container, retry, firm grip, logs.
6. How to improve a lot	The improvements ranked by impact: what you do and what the code does.
7. RealSense D435i	Whether to use it and how.
8. Training on your own demos	Short version of the fine-tuning plan.
9. When something goes wrong	Messages you may see and what to do.

The short version. Start the arman1 Studio on lightning.ai, plug in the robot (12 V) and the cameras, then in the Terminal app: `python do_task.py --check`, `python teach_drop.py box`, and `python do_task.py --pick "paper ball" --into box`. Use soft or long-flat objects and a wide, low container.

1. Status: is everything ready?

Yes for everything that can be tested without the robot, the cameras and the GPU. The table says exactly what was tested and how. The parts marked yellow can only be confirmed on the real hardware; the first real run checks them automatically and writes a log that Claude can read to fine-tune the numbers.

Part	State	How it was tested
Cloud update before every run (code sync, restart if changed, version/model/GPU checks)	Ready, tested	Against a simulated Studio folder: old files replaced, server restarted only when its code changed, missing model caught.
Whole run: model turns, grab watch, carry, retry, going home	Ready, tested	Offline end-to-end test: real LeRobot client and server code, scripted fake model, simulated gripper that stops on an object. Three scenarios pass repeatedly (tests/offline_run_test.py).
Firm grip setting and motor temperature stop	Ready, tested	Against a fake motor bus: writes, restore to normal, hot-motor stop.
RealSense colour camera path	Ready, tested without the camera	A fake sender pushed a pure red picture through the same route; it arrived red (colours not swapped).
Sharper images (378 x 378 instead of 224)	Ready, needs the GPU	Unit-tested on the Mac; confirmed on the next --check with the Studio running.
Grab-detection numbers (when 'holding' vs 'missed')	Educated guesses	Need one real run; outputs/runs/ shows the real gripper numbers. grip_test.py measures them directly.
RealSense on this Mac	Needs the camera plugged in	sudo .venv/bin/python realsense_bridge.py --test
Drop pose for each container	You teach it once	python teach_drop.py box (cup, bowl, notebook...)

Safety kept from before: 15-degree limit per move, stop if a joint is blocked for 1.5 s, stop on a motor error or a motor at 60 degrees C, hold the pose when a run ends and ease home before the motors go limp. Never 12 V on the joystick. Emergency stop: unplug the robot's power supply.

To get the old behaviour back: in settings.env set AUTO_DROP=0, RETRIES=0, GRIP_STRENGTH=normal. The complete code before these changes is in saved_setup/code_before_assist/.

2. Your documents and files

Everything lives in **workplace/so101-hackathon/**. Run every command from that folder in the **Terminal app** (macOS only lets Terminal use the cameras), after `source .venv/bin/activate`.

Documents

File	Use it for
docs/SO101_Task_Playbook.pdf	This guide: tasks, setup order, what the code does, troubleshooting.
docs/SO101_MolmoAct2_Guide.pdf	Camera positions with drawings and photos, the image fix, the full fine-tuning plan.
settings.env	All settings: ports, cameras, task, Studio address, model, AUTO_DROP, RETRIES, GRIP_STRENGTH.
outputs/runs/<time>/	One folder per run: events.txt (what happened + RESULT), log.csv (every move), camera pictures.
outputs/do_task_cam0.jpg, cam1.jpg	What the cameras see, saved by every check.

Commands

Command	What it does
python do_task.py --check	Updates the cloud code, loads the model, checks robot and cameras, saves camera previews. Moves nothing.
python do_task.py --pick X --into Y	Runs a task (also --onto Y). Builds the sentence the way the model was trained.
python do_task.py "full sentence"	Runs a task with your own wording. Add --seconds 90 to allow longer, --drop box to choose the drop pose.
python do_task.py --fake	Tests the whole cloud chain with a fake robot (Studio must be running).
python teach_drop.py box	With the joystick: teaches 'above_box', where the code drops things. Once per container.
python grip_test.py above_box	Measures the gripper empty and with each object (saves grip.json); lets you feel the grip.
sudo .venv/bin/python realsense_bridge.py	Gives the RealSense colour picture to the other programs (own Terminal tab). Add --test for one photo.
python joystick.py	Drive the robot with the joystick (for checking, not needed for tasks).
python record.py 10	Record 10 demonstrations with the joystick for training.
scripts/train_molmo.sh	Copy the demos to the Studio and train there (see section 8).
scripts/save_setup.sh	Back up calibrations, offsets, poses and settings (restore: add 'restore').
cd tests && ../.venv/bin/python offline_run_test.py retry_then_carry	Re-run the offline test after any code change (no hardware).

Settings you may change (settings.env)

Line	Meaning
FRONT_CAM=iphone	Top camera. iphone, webcam, realsense or a number.
SIDE_CAM=webcam	Side camera. Recommended: realsense (see section 7) or webcam.
MOLMO_TASK="..."	Task used when you run do_task.py without a task.
MOLMO_CHECKPOINT=	Empty = base model. After training: the path train_molmo.sh printed.
AUTO_DROP=1	Code carries a held object to the taught drop pose. 0 = model does everything.
RETRIES=2	New tries after a missed grab. 0 = none.
GRIP_STRENGTH=strong	Gripper keeps more force while squeezing. normal = LeRobot's setting.

3. Setup, step by step

A. Before the hackathon table (no hardware needed)

- **Rubber on the gripper.** Rubber bands around both fingers, or a strip of bike inner tube, anti-slip mat or hot-glue bumps on the inner faces. Rubber holds roughly 2 to 4 times better than smooth plastic with the same squeeze. 10 minutes.
- **Objects:** pick from section 4 (soft or long-flat, under about 100 g, at most two-thirds of the gripper opening).
- **Container:** wide and low (box, bowl, basket, plate, notebook). No tall narrow cup.
- **Tape marks:** a square where the object starts (15 to 27 cm in front of the robot base) and a mark for the container.

B. Power and cables

- Robot (follower): its **12 V** supply. Joystick (leader): **5 V** only, never 12 V.
- Both USB cables into the Mac. The check reads the voltage and refuses to run below 10 V.

C. Cameras

- **Top (FRONT_CAM=iphone):** iPhone 60 cm in front of the robot, 50 cm above the table, tilted about 50 degrees down, landscape, in focus. Mac Bluetooth and Wi-Fi ON, iPhone locked and near the Mac.
- **Side (SIDE_CAM=realsense or webcam):** 40 to 45 cm to the side of the object area, lens about 20 cm high, looking about 20 degrees down. For the RealSense, first start the bridge in its own Terminal tab (it asks for your Mac password):

```
sudo .venv/bin/python realsense_bridge.py
```

- Nobody standing behind the robot in view. Even light (a lamp from the side helps). Drawings: next page.

D. The cloud (Lightning AI)

- On lightning.ai open the Studio **arman1**, press **Start**, wait until it shows **Running** (GPU L4).
- Nothing else to do there. do_task.py updates the cloud code by itself **before** loading the model (section 5). Never paste Lightning's setup command (it contains your private key) anywhere except your own Terminal.
- When you are finished for the day, press **Stop** on the Studio to save credits.

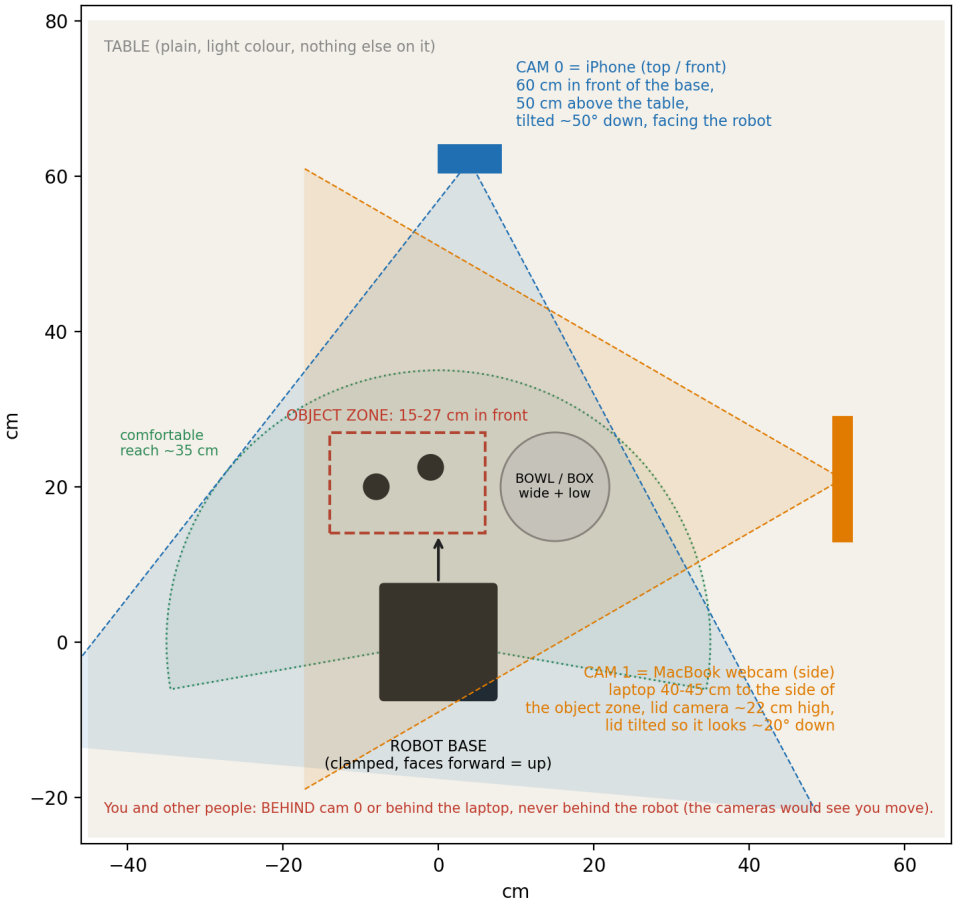
E. Check, teach, run

```
source .venv/bin/activate
python do_task.py --check # 2-6 min the first time (model loading)
open outputs/do_task_cam0.jpg outputs/do_task_cam1.jpg # do the views match the drawings?
python teach_drop.py box # once per container, with the joystick
python grip_test.py above_box # optional, 2 min: measures your gripper
python do_task.py --pick "paper ball" --into box
```

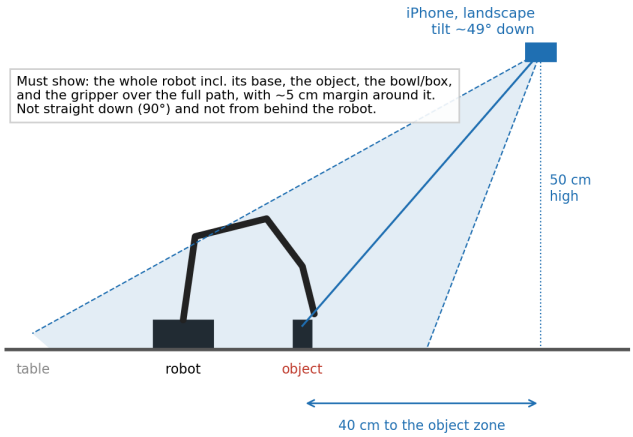
After each run the Terminal shows **RESULT** and the log folder. If it failed, tell Claude the folder name: the log shows exactly where (missed, slipped, blocked) and the camera pictures show what the model saw.

Camera positions (from the first guide)

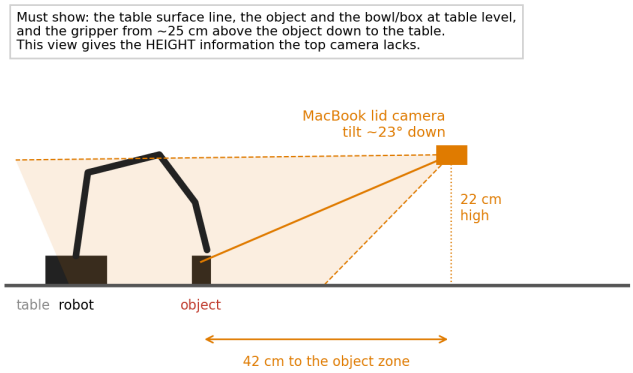
1. Top view: where everything goes



2a. Cam 0 (iPhone) seen from the side



2b. Cam 1 (MacBook webcam) seen from the front



4. The best tasks

4.1 What the model was trained and tested on

Training: the SO-100/101 version of MolmoAct2 learned from about 38,000 demonstrations in 1,222 public SO-100/101 datasets that people uploaded to Hugging Face, mostly pick-and-place. The paper does not list them one by one.

Tested by its makers on an SO-100 arm, without extra training (15 tries per task; the score gives part credit: 0.25 for reaching, 0.5 for picking up, 1.0 for placing, so these are not pure success rates):

Task	MolmoAct2	pi0 (comparison)	What it tells us
Pen onto a notebook	86.7	80.0	Long thin object lying flat + big flat target: best case.
Tissues into a basket	73.3	20.0	Soft objects forgive a weak, imprecise grip.
Fork onto a plate	70.0	30.0	Long flat object, big low target.
Block into a box	33.3	90.0	Hard small objects are its weak spot.
Stacking blocks	20.0	6.7	Precise placing: avoid.

The model page's own example: "move the arm towards the lemon, grasp it, lift it up, and drop it into the red bowl" (two cameras: top and side). do_task.py builds exactly this kind of sentence from --pick and --into.

A warning from someone else's test: a person who tested this model offline on SO-100 setups it had never seen found its movements look right but land in the wrong place on the table; it did well only on setups that were in its training data. That is why the code now takes over the carry, and why training on your own demos (section 8) is the big jump.

4.2 Ranked tasks for your gripper

#	Task	Command	Why
1	Pen or marker onto a notebook	--pick pen --onto notebook	Best tested score. Grip it in the middle.
2	Paper ball into a box	--pick "paper ball" --into box	Squishes, cannot roll far, big target.
3	Tissue or napkin into a basket	--pick tissue --into basket	Second-best tested task; soft.
4	Fork or spoon onto a plate	--pick fork --onto plate	Tested 70; flat and long.
5	Lemon, lime or plastic fruit into a bowl	--pick lemon --into bowl	The model page's own example.
6	Sponge into a bowl or box	--pick sponge --into box	Soft, light, easy to see.
7	Small plush toy into a basket	--pick "plush toy" --into basket	Soft; bright colour helps.
8	Sock or cloth into a basket	--pick sock --into basket	Soft; no exact grip needed.
9	Rubber duck into a wide bowl	--pick "rubber duck" --into bowl	Light; use a wide bowl, not a cup.
10	Wrapped candy into a bowl	--pick candy --into bowl	Light, flat.
11	Sticky-note pad onto a notebook	--pick "sticky notes" --onto notebook	Flat, parallel sides.
12	Eraser or glue stick into a pencil box	--pick eraser --into "pencil box"	Open, wide box.
13	Toy car onto a plate	--pick "toy car" --onto plate	Flat sides.

Medium (works sometimes)	Why it is harder
Cube or Lego block into a box or bowl	Common in uploaded demos, but only 33 in the paper's test; hard and small.
Ball into a bowl	Rolls away when dropped badly or pushed.
Dice or bottle cap into a box	Small; thin objects may read like 'closed on nothing' (grip_test.py shows it).
Chess pawn into a WIDE bowl	Small, round, tips over. Into a cup is much harder.

Avoid for the demo	Why
Stacking anything	20 in the paper's test.
Tall narrow cups as the target	The drop has to be precise; a wide container forgives.
Red Bull can	Shiny (confuses cameras) and heavy for this gripper (about 270 g full).
Picking up a cup or glass	Big, hard, see-through or shiny.
Two steps in one sentence	The model was trained on one pick-and-place per sentence.

4.3 Demo combos that look impressive

Several easy commands in a row look like one smart task. Teach each container once (teach_drop.py).

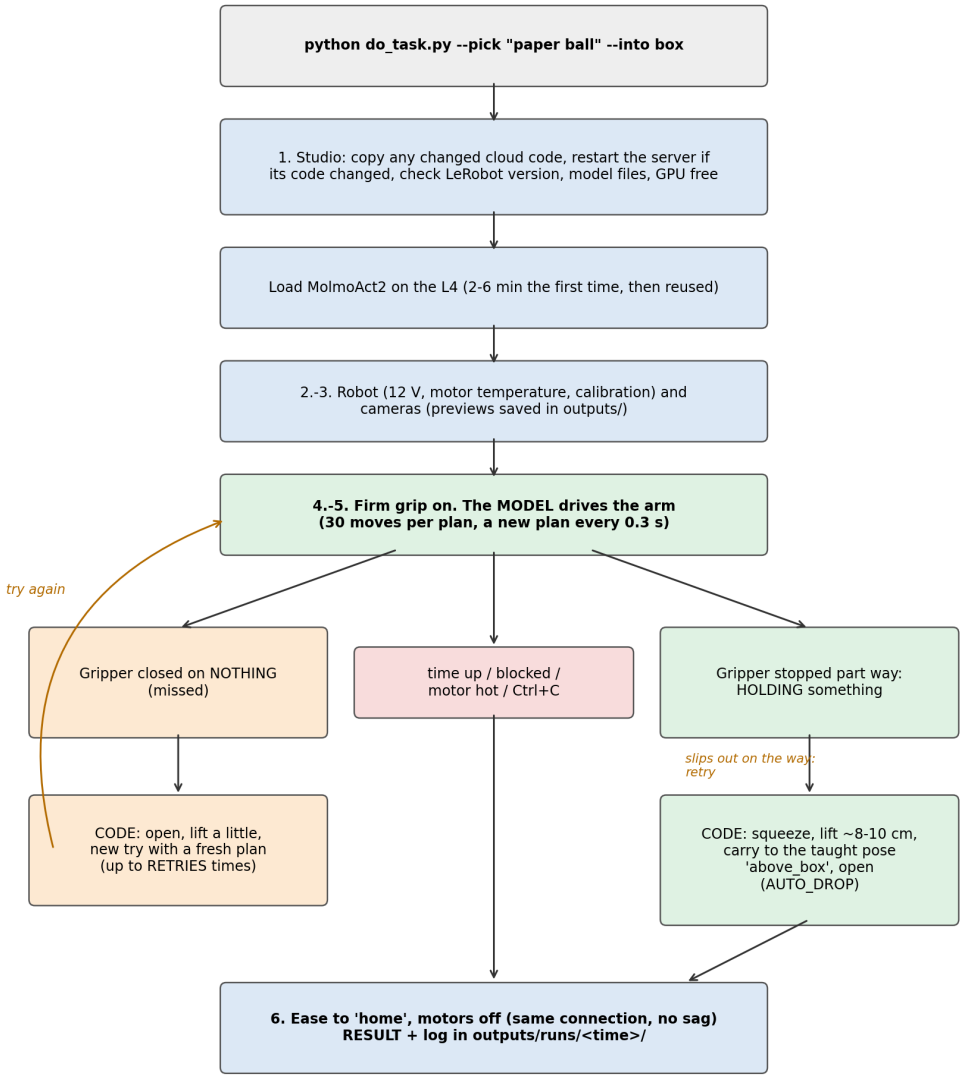
```
# "Clean the desk"
python do_task.py --pick pen --onto notebook
python do_task.py --pick "paper ball" --into box
python do_task.py --pick fork --onto plate

# "Breakfast"
python do_task.py --pick lemon --into bowl
python do_task.py --pick spoon --onto plate
```

4.4 Wording rules

- Use the trained pattern (--pick/--into does it): "move the arm towards the X, grasp it, lift it up, and drop it into the Y".
- Name colours when there is more than one similar thing: "the red block", "the white pawn".
- One object, one target per command. Keep the same words every time for the same task.
- The container word picks the drop pose: --into box uses 'above_box', --into "red bowl" uses 'above_bowl'.

5. What the code does for you



5.1 The cloud is updated before the model loads

- Every run first compares the cloud code files (**cloud/**) on this Mac with the Studio (by checksum) and copies what changed, then checks the copy arrived. If the model server's own code changed, the old server is stopped and started again with the new code.
- It then checks: the same LeRobot version on both sides (0.6.1), the model files exist on the Studio, and no training run is using the GPU.
- Only then is the model loaded (2 to 6 minutes the first time; later runs reuse the loaded model and start in seconds). If a server is still loading from an earlier start, it waits for it instead of starting over.

5.2 Help during the task (assist.py)

Feature	What happens	Switch off
Grab watch	Compares where the model tells the gripper to go with where it really is: holding, missed, slipped out, let go.	always on (only watches)
Carry (AUTO_DROP)	Once holding: squeeze fully, lift about 8 to 10 cm, move to the taught 'above_<container>', open. The container never moves, so this half always works, and slow smooth moves keep a weak grip from shaking the object loose.	AUTO_DROP=0

Feature	What happens	Switch off
Retry (RETRIES)	Closed on nothing: open, lift a little, start a fresh try. Also after the object slips out during the carry.	RETRIES=0
Firm grip	LeRobot lets the gripper drop to the motor's 20% holding force after squeezing for 2 s. 'strong' keeps 35%; the maximum (50%) is unchanged. The original value is saved and restored by 'normal'.	GRIP_STRENGTH=normal
Motor heat	The check refuses to start above 55 degrees C; a run stops at 60 (motors cut out at about 70).	always on
One connection	Before, going home reconnected the robot, which switches all motors off for a moment: the arm sagged and dropped what it held. Now one connection for the whole run.	always on
Run log	outputs/runs/<time>/: events.txt, log.csv, camera pictures per try.	always on

6. How to improve a lot (biggest first)

#	Improvement	You do	Code does	Expected effect
1	Train on your own demos	50 joystick demos (5 spots x 10), about 1 to 1.5 h	scripts/train_molmo.sh trains on the L4	Biggest: it learns your table, cameras and gripper. 70-90% on one fixed task is typical, not guaranteed.
2	Code carries and drops	Teach the drop pose once	Carry after a good grab (done)	Removes the failures after a good grab.
3	Retry missed grabs	Nothing	Done (RETRIES=2)	A miss no longer wastes the run.
4	Easy task first	Paper ball / tissue / pen	--pick/--into wording (done)	Probably the quickest gain, no training.
5	Rubber on the gripper	10 minutes with rubber bands	Firm grip setting (done)	Fewer slips, especially smooth objects.
6	Same scene every time	Tape marks, same lamp, empty table	Nothing	The model reacts to everything it sees.
7	Cameras as in the drawings	Place, focus, check previews	378 x 378 images (done)	Better distance judgement, fewer missed grabs.

Why the gripper is weak, in one line: the motor's overload protection (set by LeRobot to protect the motor) cuts the force after 2 s of squeezing, and smooth plastic fingers slide. Rubber fixes the sliding; GRIP_STRENGTH=strong softens the cut.

7. RealSense D435i

- **Works, with one extra step.** On a Mac, normal apps only get its grey infrared picture. Its colour camera needs Intel's library, which on macOS 12 and newer needs admin rights (sudo). realsense_bridge.py does that and passes the colour picture to the other programs, only inside this Mac.
- **Depth does not help this model:** this MolmoAct2 version uses only the colour pictures (its depth option is switched off).
- **Sharpness does not decide it:** every picture is shrunk to 378 x 378 before the model sees it.
- **The real advantage is placement:** it is a separate camera on a cable, so it can stand exactly where the drawings say. The MacBook webcam forces the whole laptop into that spot. The model page's example also used two RealSense cameras (top + side).
- **Recommendation:** RealSense as the **side** camera (SIDE_CAM=realsense), iPhone on top. If the iPhone keeps dropping out or looking blurry, put the RealSense on top instead. Plug it straight into a USB-C port (USB 3), not a hub.

```
sudo .venv/bin/python realsense_bridge.py --test # one colour photo:
outputs/realsense_test.jpg
sudo .venv/bin/python realsense_bridge.py # leave running in its own tab, then run
do_task.py
```

8. Training on your own demos (short version)

- **Record:** python record.py 10, five times, moving the object to 5 spots (10 demos each), same task sentence, same cameras, same light. Smooth, decisive demos: grip low on the object, lift, carry, open over the container.
- **Train:** scripts/train_molmo.sh (action-expert mode, fits the L4). Turn off auto-sleep on the Studio while it trains. One round of 3,000 steps: about 2 to 4 hours on the L4 (estimate; the real speed shows in the first minutes).
- **Use it:** set MOLMO_CHECKPOINT in settings.env to the path it prints; do_task.py restarts the server with the new model.
- **Rounds:** expect 2 to 3 rounds (record, train, test 20 tries, add demos where it fails): 1 to 2 days, about 6 to 12 GPU hours. Full plan in docs/SO101_MolmoAct2_Guide.pdf.

9. When something goes wrong

You see	Do this
the 'arman1' Studio is stopped	lightning.ai: open arman1, Start, wait for Running, run again.
Permission denied (publickey)	Run Lightning's SSH setup command again yourself (it contains your private key).
a training run is using the GPU	Wait for training to finish or stop it (the message shows the command).
model not found on the Studio	MOLMO_CHECKPOINT in settings.env: empty for the base model, or the exact trained path.
model server crashed	Tell Claude; the log is ~/molmo/server.log on the Studio.
robot motors get only X V	Plug in the robot's 12 V supply (never 12 V on the joystick).
motor is hot / STOP: motor too hot	Switch the robot off for 10 minutes.
no 'iphone' camera found	Mac Bluetooth and Wi-Fi on, iPhone locked and close; wait a few seconds; run again.
no picture from the RealSense	Start sudo .venv/bin/python realsense_bridge.py in another tab first.
auto drop off for this run	Teach the drop pose it names: python teach_drop.py box.
STOP: the robot cannot reach its target	Something blocks the arm. Clear it; check the cable is not caught.
missed again and again	Object position, light and camera views first; then send Claude the outputs/runs folder.
holding detected but nothing in the gripper	Thin object or the jaws touched the table: run grip_test.py and send the log to Claude.
object slips during the carry	Rubber on the fingers; lighter object; GRIP_STRENGTH=strong.

Sources

- MolmoAct2 paper, arXiv 2605.02881 (SO-100/101 zero-shot table, training data): arxiv.org/html/2605.02881
- Model page: huggingface.co/allenai/MolmoAct2-SO100_101 (example task, two cameras, depth disabled)
- Community out-of-the-box test: mcobzarenco-fontaine-blog.static.hf.space/posts/2026-08-10-molmoact2-oob-results.html
- Feetech STS memory table (overload and holding torque): aifitlab-wiki.super.site/feetech-servo-motor-docs/feetech-sts-servo-memory-table-analysis
- Intel RealSense macOS installation (sudo on macOS 12+): dev.realsenseai.com/?p=40689
- pyrealsense2 for macOS: github.com/cansik/pyrealsense2-macosx